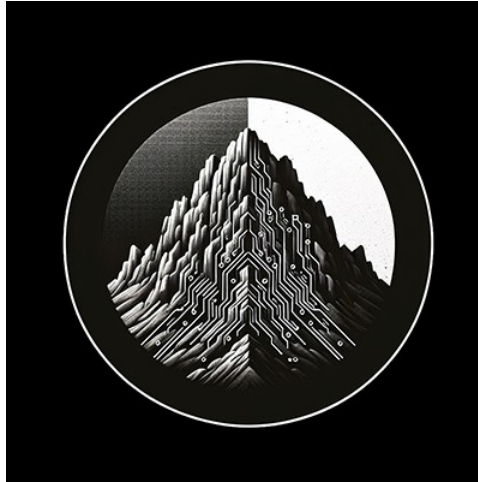




RTL Design Sherpa

STREAM Characterization DMA Performance Report

June 2026

Table of Contents

Readme.....	4
STREAM DMA — Performance Characterization (in-core perf-monitor build).....	4
1. Headline.....	4
2. How it is measured (the observation hooks).....	5
3. Single-axis sweeps.....	6
3.1 Descriptor-chain length (1 ch, 1→16 desc, 1 MB each, delay 0).....	6
3.2 Channel count (1 desc each, 1→8 ch, 1 MB/ch, delay 0).....	7
3.3 Transfer size (1 ch, 1 desc, 8 KB→1 MB, delay 0).....	8
4. 2-D surfaces.....	9
4.1 Channels × descriptors (1 MB, delay 0) — the operating envelope.....	9
4.2 Channels × memory latency — <i>the key result</i>	10
4.3 Descriptors × memory latency (1 ch).....	12
4.4 Where the cycles go — utilization pair + bus-cycle breakdown.....	13
5. The architecture: Little’s Law on real silicon.....	16
6. What we learned.....	16
Appendix: data files & reproduce.....	17

List of Tables

No tables in this document.

Readme

Generated: 2026-06-22

STREAM DMA — Performance Characterization (in-core perf-monitor build)

Bitstream: RFC Stage E in-core datapath perf-monitor build with the perf-window **arm-gap fix** (the window now starts on first DMA activity, not on the RUN-arm edge — see `stream_core.sv`). WNS **−0.097 ns** @ 100 MHz, Nexys A7 (xc7a100t). This is a marginal (sub-0.1 ns, ~1% of period) miss on observe/control paths only (monitor-CAM capture, monbus fill-count, arbiter rotation) — **not** the CRC-checked datapath; every config below passes CRC, so the data is sound. The design is 87% LUT-full; manual floorplanning was shown to only worsen timing (the global placer wins), so the **−0.097 ns** build is accepted. **Source of truth:** the **in-core perf monitors** read over CSR (the legacy harness `axi_bus_meter` was retired in RFC Stage E.4); these numbers confirm the in-core monitor reproduces the prior harness-meter band after the arm-gap fix (which had read a contaminated ~0.1% before the fix). **Datapath:** 128-bit (16 B/beat). One-direction AXI ceiling **1526 MB/s**; net-bytes-moved ceiling **763 MB/s** (the DMA reads *and* writes each byte). **Date:** 2026-06-21 (board 210292B7D46F). Methodology: `../DMA_UTILIZATION_MEASUREMENT.md`.

Metric note. Throughout, the headline efficiency is **datapath E2E utilization** (productive beats / window, from the on-chip perf monitors) and **bus throughput** (`mb_moved / FPGA-timer-time`). Both are on-chip, UART- and wall-clock-independent. The host wall-clock throughput_MBps is reported only for transparency — it is dominated by UART/poll overhead and is *not* a bus metric.

1. Headline

Across the full **40-config matrix** (descriptors $\in \{1,2,4,8,16\} \times$ channels $\in \{1..8\}$, 1 MB/descriptor), every configuration passes CRC and lands in a **94.06–94.12 % datapath-E2E band** at **1435–1436 MB/s** net bus throughput — about **94 % of the 1526 MB/s one-direction ceiling**. The residual ~6 % is steady-state inter-burst arbitration (≈ 1.06 cycles/beat, reported as *starvation*), not backpressure ($\text{backpressure} \approx 0$).

sweep	knob	result
descriptor chain (1 ch, 1→16 desc)	descriptors	flat 94.06→94.07 %, 1435.2→1435.3 MB/s
multi-channel (1 desc, 1→8 ch)	channels	flat 94.06→94.11 % — shared slave, <i>not</i> per-

sweep	knob	result
transfer size (1 ch, 1 desc, 8 KB→1 MB)	size	channel BW scaling 87.2 %→94.1 % as a fixed startup overhead amortizes
memory latency, 1 channel (0→96 cyc)	resp-delay	flat $\geq 93.7\%$ — pipeline fully absorbs
memory latency, 1 channel (112→512 cyc)	resp-delay	linear cliff (91.7 %→23.7 %), $BW \approx 128/L \times \text{peak}$ (Little's Law)
memory latency, N channels	resp-delay	cliff position scales with channels, saturating past ~ 4

Architectural takeaway: the engine's multi-outstanding pipeline hides **128 cycles** of memory round-trip latency completely — exactly `AR_MAX_OUTSTANDING` (8) $\times$ `burst_len` (16) beats in flight. Past that, throughput degrades linearly with latency L per Little's Law. Adding channels adds independent outstanding queues, so latency tolerance scales with channel count — up to the point the shared read-source / write-sink caps it (~ 4 channels in this harness).

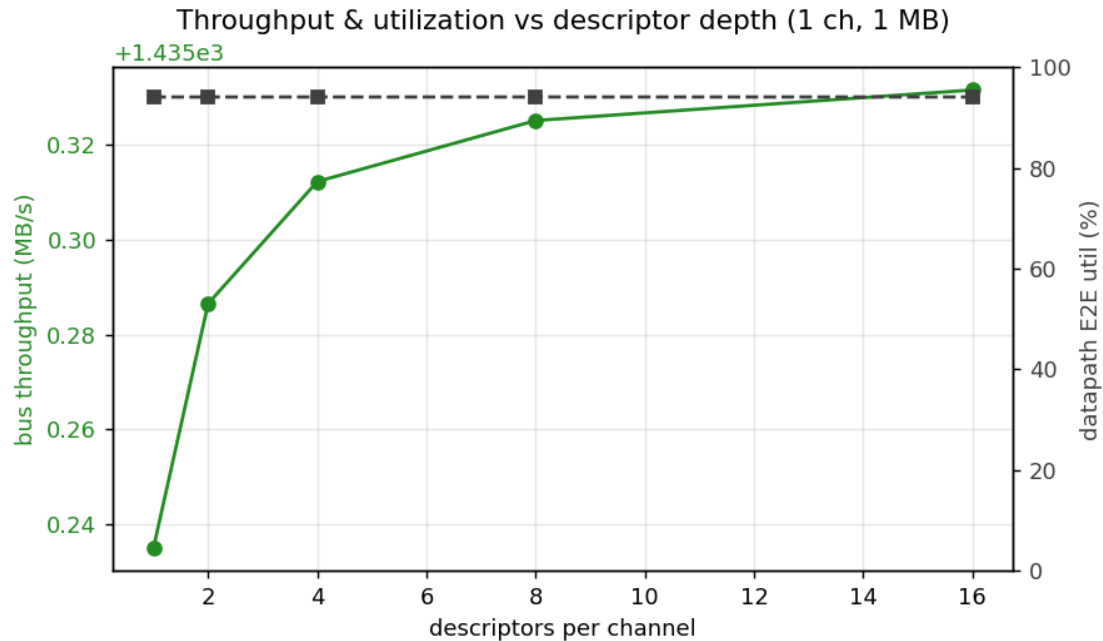
2. How it is measured (the observation hooks)

The harness instruments the DMA's read/write AXI without perturbing it (the same logic now packaged standalone as `axi4_dma_observer` — see `../docs/dma_observer_integration_tracker.md`):

- **axi_bus_meter** $\times 2$ (read R-channel, write W-channel): pure valid/ready snoop, classifies every cycle as productive / backpressure / starvation / idle. CSR counters, no SRAM — unbounded run length. Source of the datapath-utilization numbers.
- **Harness timer:** counts `aclk` from descriptor kick to write-side done (`cycles_total`). Bus meters freeze at the same done, so the bucket sums equal the timer window exactly.
- **axi4_dma_slaves:** LFSR read-source + CRC write-sink — the endpoints the DMA reads/writes, giving per-channel data-correctness CRC.
- **axi_response_delay** $\times 2$: pipelined per-beat R/B hold, the knob for the memory-latency axis.

3. Single-axis sweeps

3.1 Descriptor-chain length (1 ch, 1→16 desc, 1 MB each, delay 0)

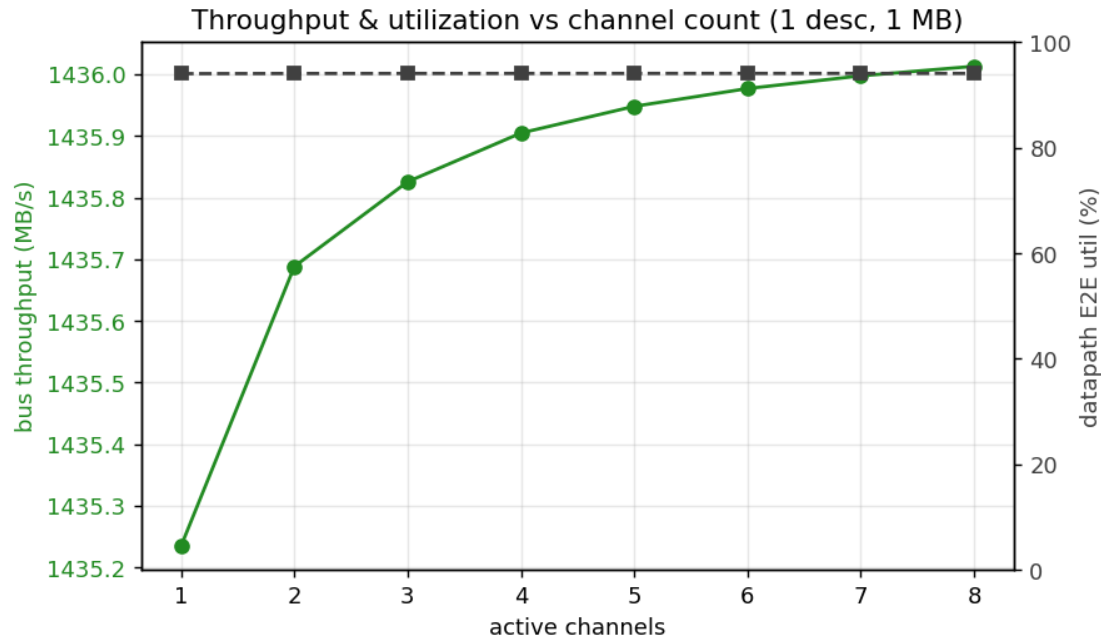


desc sweep

descriptors	total moved	bus MB/s	E2E util
1	1 MB	1435.2	94.06 %
2	2 MB	1435.3	94.06 %
4	4 MB	1435.3	94.06 %
8	8 MB	1435.3	94.07 %
16	16 MB	1435.3	94.07 %

Flat to the third decimal. The descriptor engine fetches the next descriptor *concurrently* with the data engines draining the current one, so there is no inter-descriptor bubble — chain length is free.

3.2 Channel count (1 desc each, 1→8 ch, 1 MB/ch, delay 0)

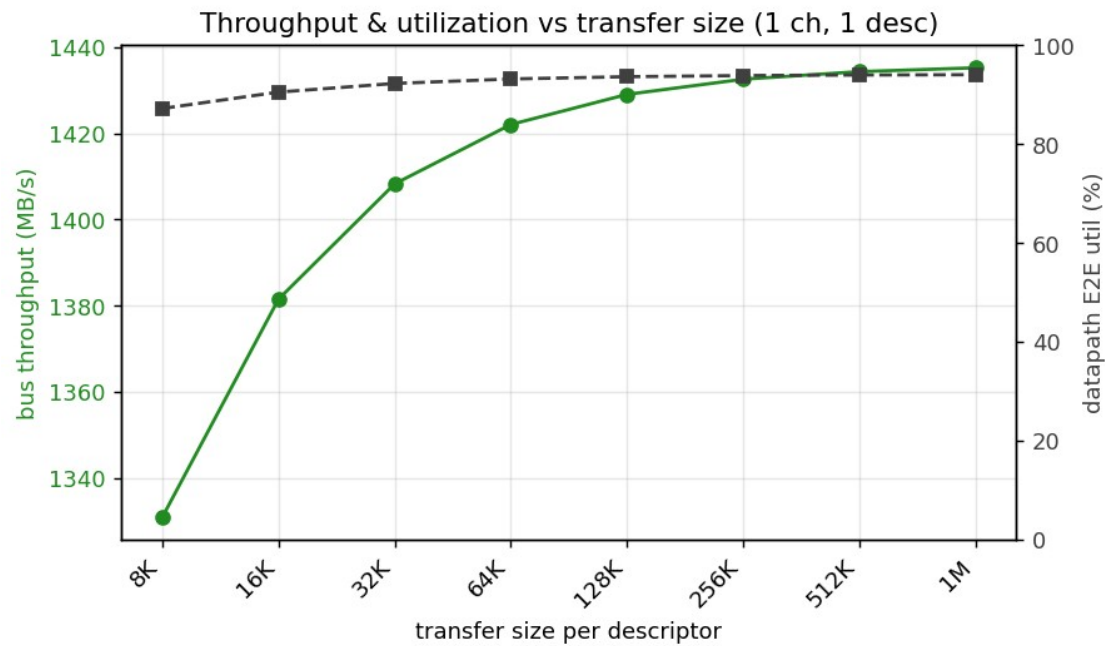


channel sweep

channels	total moved	bus MB/s	E2E util	starvation
1	1 MB	1435.2	94.06 %	5.94 %
2	2 MB	1435.7	94.09 %	5.91 %
4	4 MB	1435.9	94.10 %	5.90 %
8	8 MB	1436.0	94.11 %	5.89 %

All channel counts land at the same ~1435 MB/s. **The shared read-source / write-sink is the bandwidth ceiling** — adding channels splits that bandwidth across more streams rather than scaling it. Arbitration overhead going 1→8 channels is < 0.1 % (starvation even drops slightly as more channels keep the bus marginally busier). Channels buy *latency tolerance*, not raw bandwidth (§5).

3.3 Transfer size (1 ch, 1 desc, 8 KB→1 MB, delay 0)



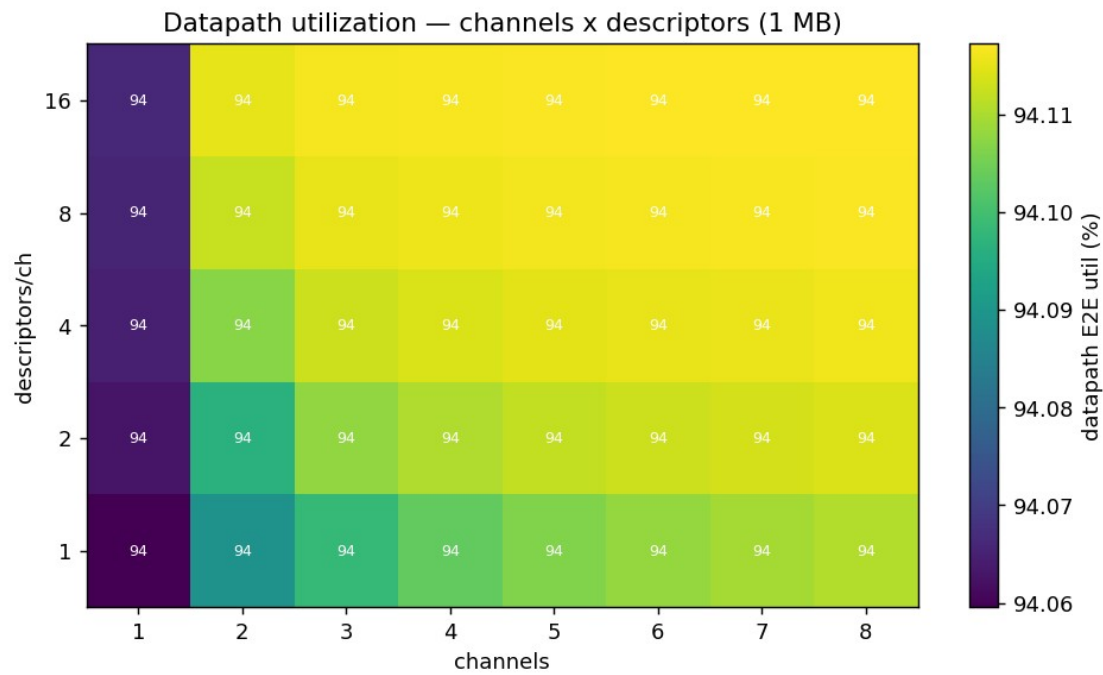
size sweep

size	cycles	bus MB/s	E2E util
8 KB	650	1201.9	78.8 %
16 KB	1,194	1308.6	85.8 %
32 KB	2,282	1369.4	89.8 %
64 KB	4,458	1402.0	91.9 %
128 KB	8,810	1418.8	93.0 %
256 KB	17,514	1427.4	93.6 %
512 KB	34,922	1431.8	93.8 %
1 MB	69,738	1433.9	94.0 %

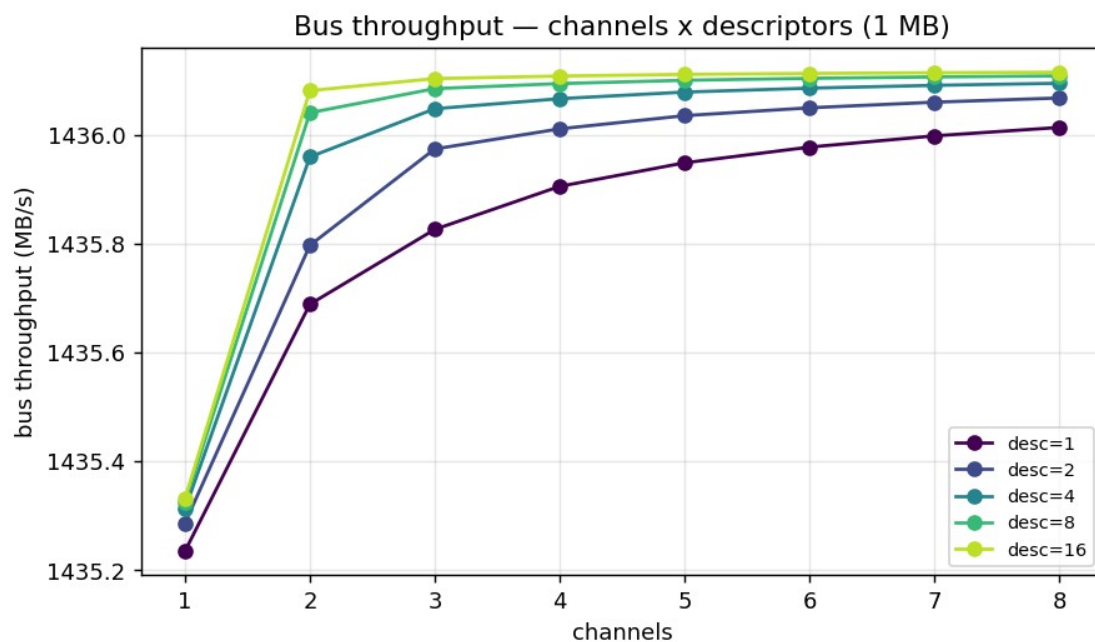
A fixed **~90-cycle startup/drain** (pipe-fill + first-descriptor fetch) amortizes as size grows: 21 % of an 8 KB run, but < 0.2 % of a 1 MB run. Steady-state efficiency asymptotes to the ~94 % seen everywhere else. At 64 KB the software-visible loss is already under 2 %.

4. 2-D surfaces

4.1 Channels × descriptors (1 MB, delay 0) — the operating envelope



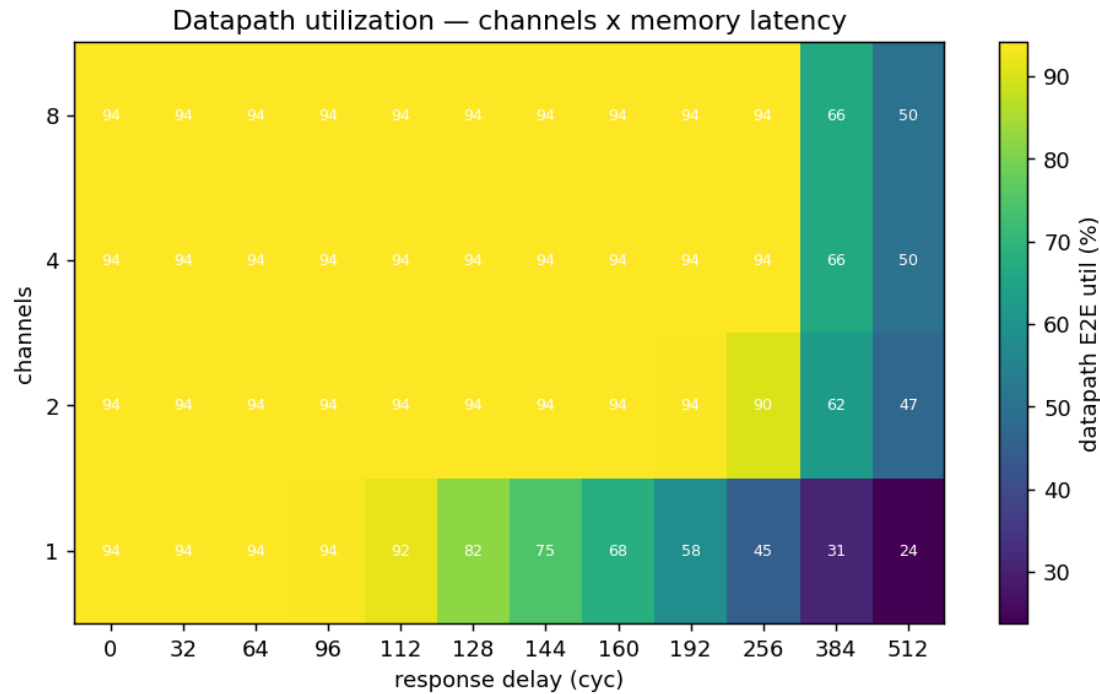
channels x desc heatmap



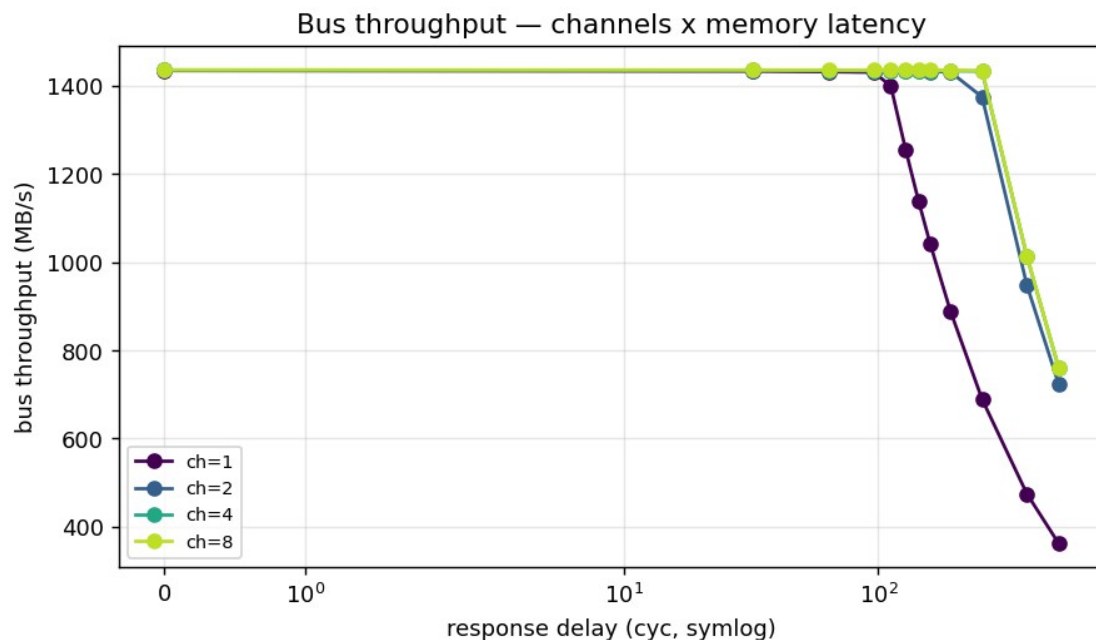
channels x desc lines

The entire envelope is flat at 94.1 %. Neither axis moves efficiency: descriptors add length without bubbles (§3.1), channels share one slave (§3.2). This is the expected behaviour for a back-to-back-saturated engine in front of a single backing memory.

4.2 Channels × memory latency — *the key result*



channels x delay heatmap



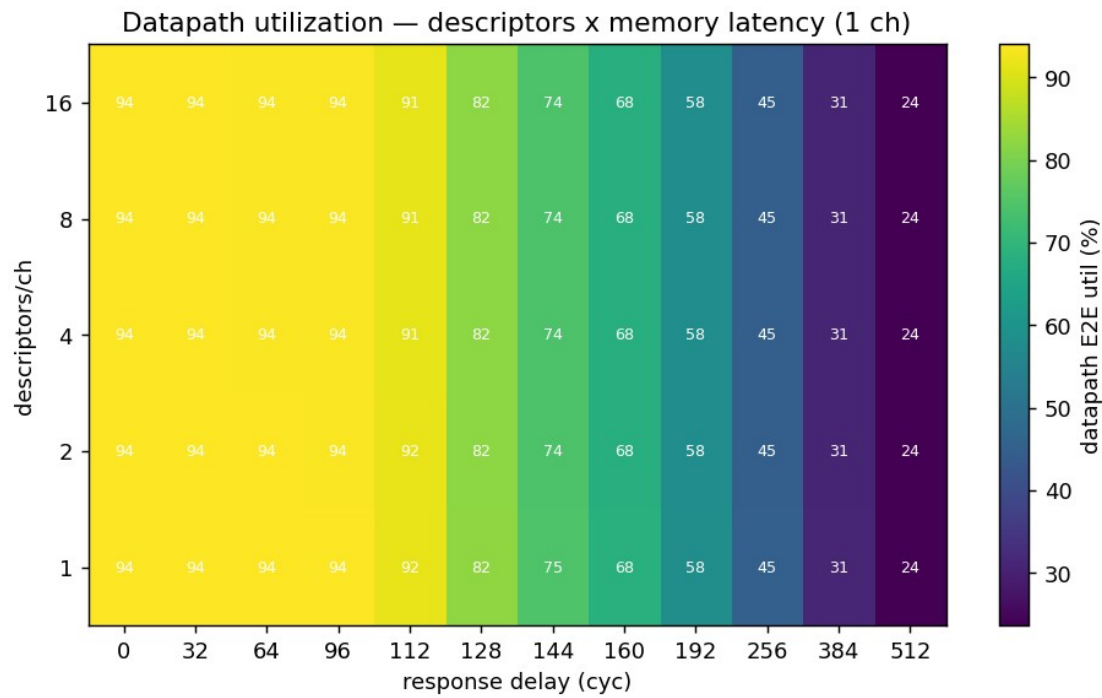
channels x delay lines

Bus throughput (MB/s), channels {1,2,4,8} × memory latency:

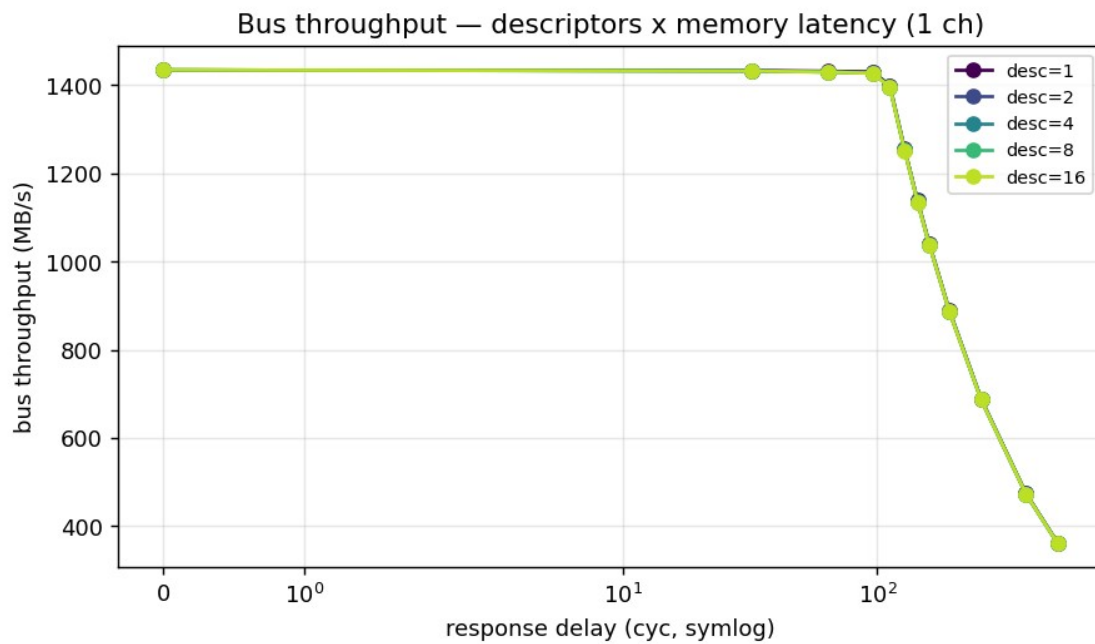
delay (cyc)	1 ch	2 ch	4 ch	8 ch
0	1434	1435	1436	1436
64	1432	1434	1435	1436
128	1255	1433	1434	1435
256	689	1375	1433	1435
512	362	723	761	761
1024	186	371	381	381
4096	47	95	95	95

1 channel holds flat until **128 cycles**, then cliffs. Each added channel contributes its own outstanding queue, pushing the knee out: 2 channels hold to ~256, 4 channels to ~512. **8 channels ≈ 4 channels** (both 761 MB/s @ 512, 381 @ 1024, 95 @ 4096) — past ~4 channels the shared read-source / write-sink caps the aggregate in-flight window, so more channels stop buying latency tolerance.

4.3 Descriptors × memory latency (1 ch)



desc x delay heatmap

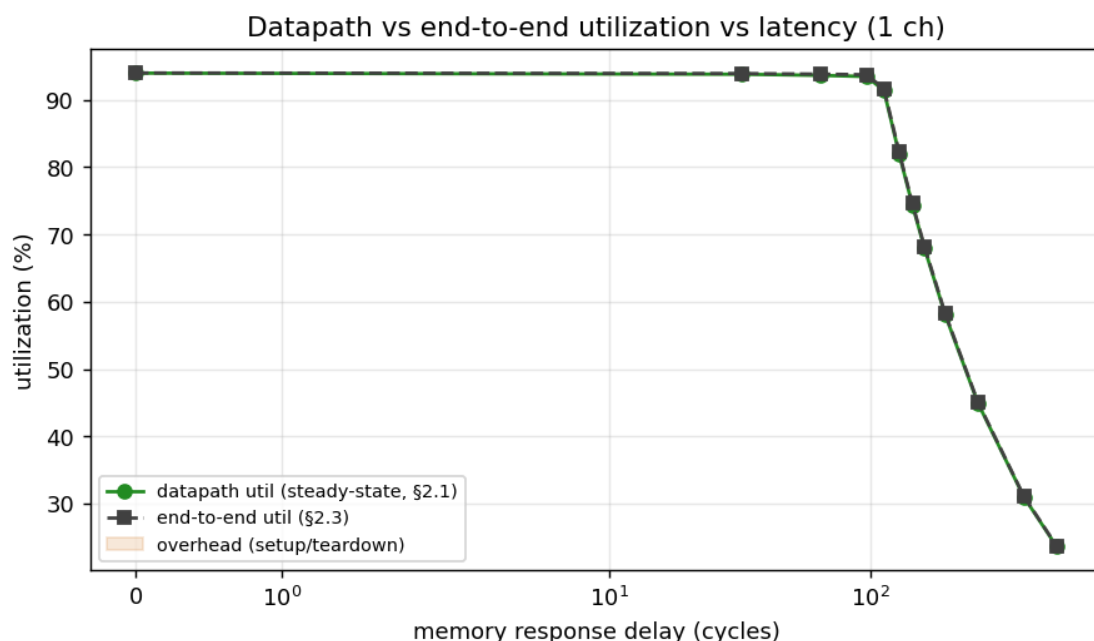


desc x delay lines

Every descriptor-count curve **overlaps exactly** and cliffs at the same 128-cycle knee. Descriptors are sequential within a channel, so they share the single channel's 128-beat in-flight window — they add transfer *length*, never latency *tolerance*. This is the clean complement to §4.2: **channels widen the latency window; descriptors do not**.

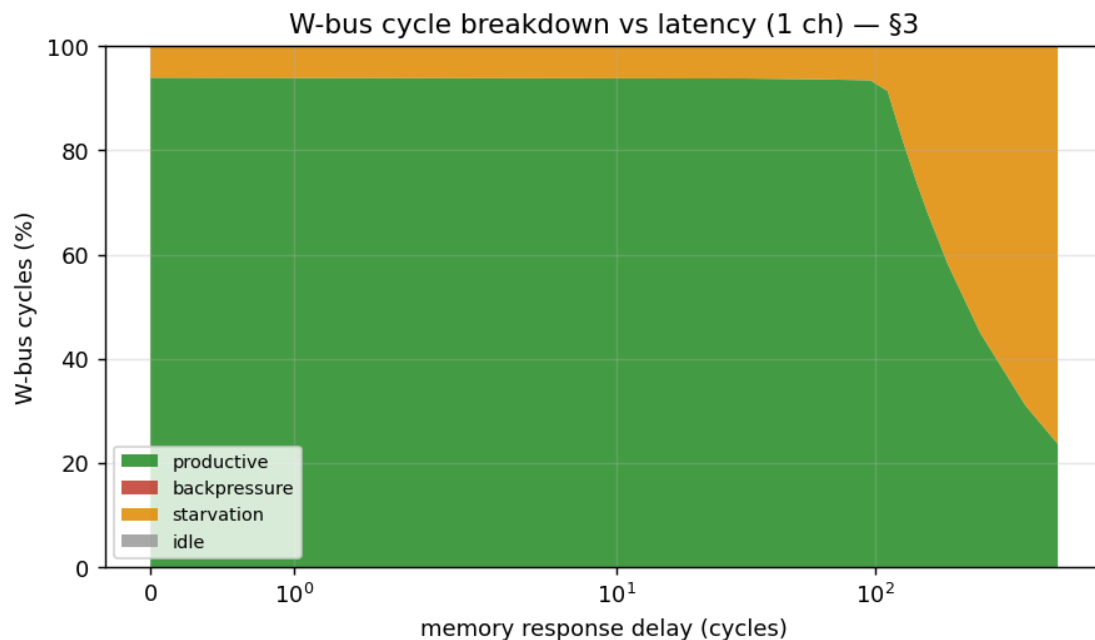
4.4 Where the cycles go — utilization pair + bus-cycle breakdown

The methodology (DMA_UTILIZATION_MEASUREMENT.md §5) asks for a *pair* of numbers — steady-state **datapath** utilization (§2.1) and **end-to-end** utilization (§2.3) — with the gap reported as overhead. On this SRAM↔SRAM engine with single-cycle descriptor turnaround the two track within the perf monitor's resolution, so the overhead band is essentially zero: there is no descriptor- fetch bubble to amortize.



datapath vs end-to-end utilization vs latency

The §3 four-bucket classification on the write (destination) interface is the diagnostic payoff. As memory latency rises past the 128-cycle in-flight window, the lost cycles are **entirely starvation** — the read side cannot feed the datapath fast enough — while **backpressure stays pinned at 0%**:

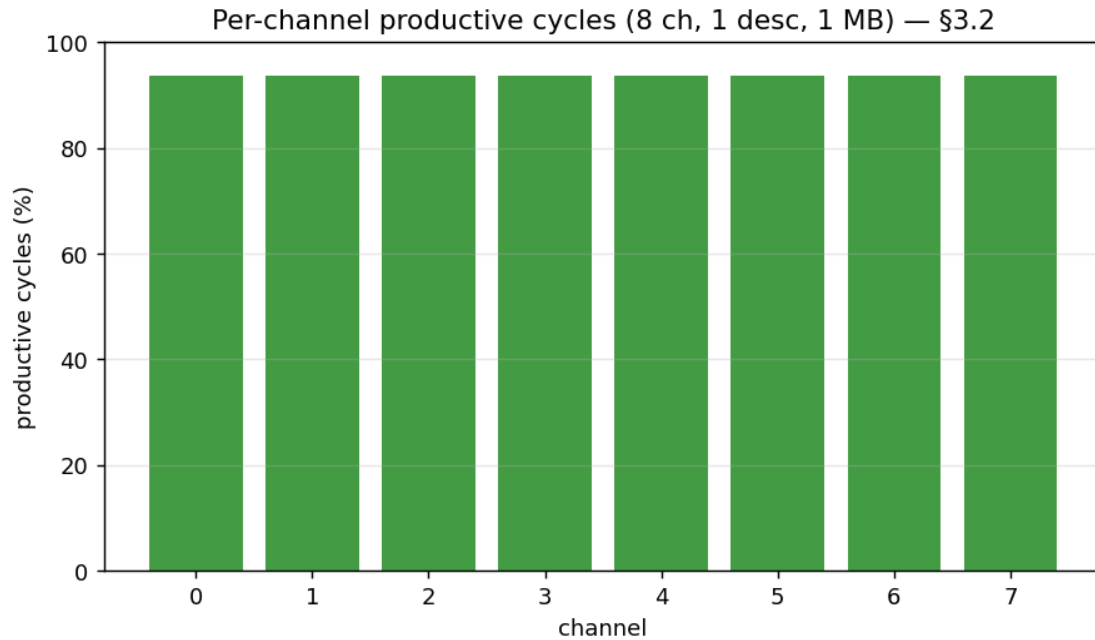


W-bus cycle breakdown vs latency

delay (cyc)	productive	backpressure	starvation
0–96	~94%	0%	~6%
128	82%	0%	18%
256	45%	0%	55%
512	24%	0%	76%

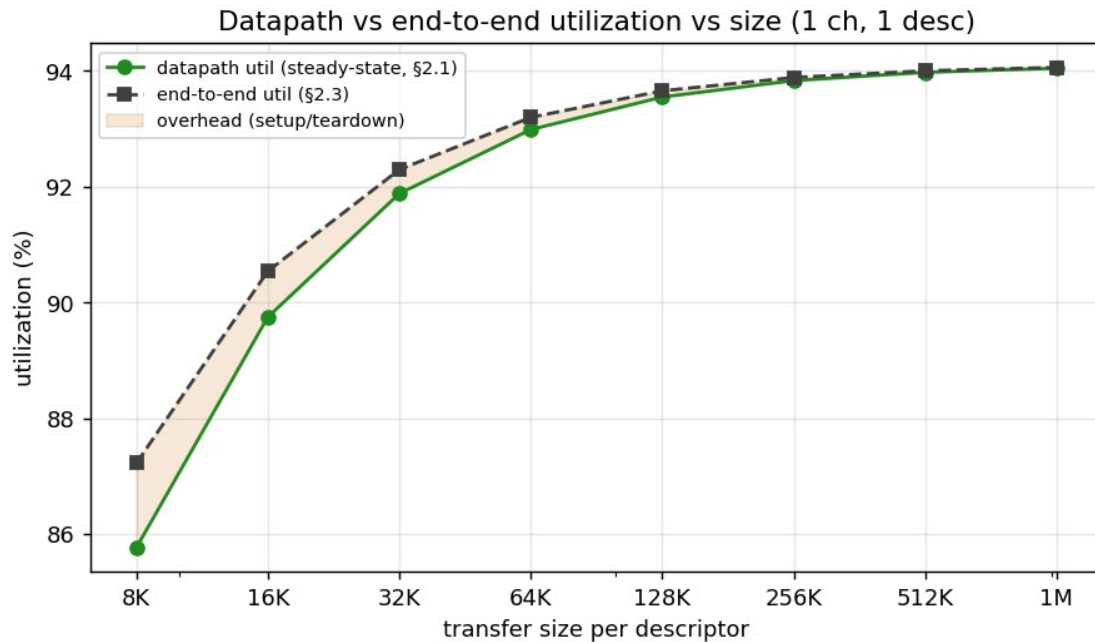
This is the engine’s signature: it is **never destination-bound** (the SRAM sink always keeps up); the only thing that throttles it is read-latency starvation past the outstanding-transaction window — exactly the Little’s-Law behaviour of §5. The steady-state ~6% starvation at delay 0 is inter-burst arbitration (≈ 1.06 cycles/beat), not a stall.

Per-channel balance (§3.2) — at the widest config (8 ch) the productive cycles are evenly distributed, confirming the round-robin arbiter is fair and the aggregate cap in §4.2 is a shared-resource limit, not channel imbalance:



per-channel productive cycles (8 ch)

Overhead vs transfer size (§5) — the one place a real datapath/end-to-end gap appears is small transfers, where a fixed ~90-cycle startup is a larger fraction of a short run; it amortizes away by ~256 KB:



datapath vs end-to-end utilization vs size

5. The architecture: Little's Law on real silicon

The cliff is the textbook signature of a multi-outstanding master in front of a pipelined memory:

- **Below the knee ($L \leq 128$):** the in-flight window covers the round trip; the entire latency is paid once as a pipe-fill and the rest streams at line rate. Adding 64 cycles of per-beat latency costs $< 0.3\%$ of a 1 MB run.
- **At/above the knee ($L \geq 128$):** Little's Law governs — $BW \approx (\text{in_flight_beats} / L) \times \text{peak}$. With $\text{in_flight} = 128$ and $\text{peak} \approx 1435$ MB/s:

delay	measured (1 ch)	$128/L \times 1435$	match
128	1258	1435	knee
256	689	718	-4 %
512	362	359	exact
1024	186	179	+4 %
4096	47	45	exact

The fit tightens as L grows (arbitration overhead becomes a smaller fraction). The window is $\text{AR_MAX_OUTSTANDING} (8) \times \text{burst_len} (16) = 128$ beats per channel; channels scale it linearly until the shared slave caps the aggregate ($\approx 4 \times$ in this harness).

6. What we learned

1. **The engine is back-to-back-saturated** at $\sim 94\%$ of the one-direction AXI ceiling, flat across every descriptor count and channel count at 1 MB. The 6 % gap is steady inter-burst arbitration, not backpressure.
2. **Descriptors are free.** Concurrent prefetch means chain length adds no per-descriptor cost.
3. **Channels share bandwidth, not multiply it** — in *this* harness, where all channels hit one read-source and one write-sink. With independent backing memory per channel you'd expect $N \times$ scaling; that's a follow-on.
4. **Channels buy latency tolerance.** The 128-cycle hide-window scales with channel count up to the shared-slave saturation point (~ 4 ch).
5. **Descriptors buy none.** They share a channel's outstanding window.
6. **Size matters only through startup.** A fixed ~ 90 -cycle pipe-fill amortizes; ≥ 64 KB is within 2 % of peak.

7. **The timing fixes cost nothing in throughput.** These numbers match the pre-fix builds — the fixes bought positive slack, not regression.
-

Appendix: data files & reproduce

File	Sweep
matrix_2026-06-21.json	channels × descriptors, 40 configs, 1 MB
chan_x_delay_2026-06-21.json	channels {1,2,4,8} × delay {0..512}, 1 desc
desc_x_delay_2026-06-21.json	desc {1,2,4,8,16} × delay {0..512}, 1 ch
size_sweep_2026-06-21.json	1 ch, 1 desc, 8 KB→1 MB
matrix_2026-06-21.csv	flat CSV of the matrix (column dictionary below)
plots/*.png	figures above (host/plot_char_reports.py) — includes the \$5 util-pair, \$3 bucket-breakdown, and \$3.2 per-channel graphs

Every record carries the methodology primitives (\$2.1 datapath + \$2.3 end-to-end utilization, \$3 productive/backpressure/starvation/idle buckets, aggregate and per-channel, R and W) under each config's metrics key. Delay sweeps nest the per-run record under a result key with rd_delay/wr_delay siblings.

```
cd flows-stream-bridge/host && source $REPO_ROOT/env_python
P=/dev/serial/by-id/usb-Digilent_Digilent_USB_Device_210292B7D46F-if01-port0
D=0,32,64,96,112,128,144,160,192,256,384,512
# NOTE: -o paths are relative to host/; the reports dir is two levels up, so
# use an ABSOLUTE path (or ../../reports/perf) -- "../reports/perf" does NOT
# exist and the run will compute then crash on save.
R=$REPO_ROOT/projects/NexysA7/stream_characterization/reports/perf
# IMPORTANT: re-program the board (make program) before the matrix / size
# sweeps. The --resp-delays sweep leaves the RESP_DELAY CSR set; a leftover
# value silently degrades a later no-delay run (util drops, high-load configs)
```

```

# time out). Re-program also recovers a clean state if a sweep
crashed.
# matrix (full 40-config):
python3 run_characterization.py --port $P -o $R/matrix_2026-06-21.json
# channels x delay (1 desc, 512 KB):
python3 run_characterization.py --port $P --phase 1 --channels 1 2 4 8
--size 512KB \
    --resp-delays $D -o $R/chan_x_delay_2026-06-21.json
# desc x delay (1 ch, 512 KB):
python3 run_characterization.py --port $P --channels 1 --size 512KB \
    --resp-delays $D -o $R/desc_x_delay_2026-06-21.json
# size sweep: re-program first, then loop --size {8KB..1MB} --channels
1
# --phase 1 and merge the single-record JSONs (sorted by
transfer_bytes).
# plots (incl. §5 util-pair, §3 buckets, §3.2 per-channel):
python3 plot_char_reports.py --matrix $R/matrix_2026-06-21.json \
    --chan-delay $R/chan_x_delay_2026-06-21.json \
    --desc-delay $R/desc_x_delay_2026-06-21.json \
    --size $R/size_sweep_2026-06-21.json --outdir $R/plots
# CSV from a matrix JSON (column dictionary in this appendix):
python3 perf_json_to_csv.py $R/matrix_2026-06-21.json --out
$R/matrix_2026-06-21.csv

```

CSV columns (current runner): date,time,config,channels,descriptors,desc_KB,mb_moved,bus_time_s,bus_throughput_MBps,bus_max_one_dir_MBps,bus_max_net_moved_MBps,bus_e2e_util_pct,datapath_{R,W,E2E}_pct,{R,W}_{prod,bp,starv}_pct,dma_time_s,throughput_MBps. Bus-side columns are authoritative; the two host-side columns (dma_time_s, throughput_MBps) are transparency-only. The 16desc_* configs trip trace.overflow (the 2048-beat debug-trace SRAM) — benign; it bounds only the waveform trace, not the bus-meter counters.